Detección de documentos casi duplicados a gran escala

MinHash, SuperMinHash y FracMinHash con LSH

Vicente Hernández Vicente Wiederhold

Análisis de Grandes Volúmenes de Datos

16 de octubre de 2026

El problema

- Las colecciones masivas de documentos contienen **copias casi idénticas** (noticias sindicadas, plantillas, *boilerplate*, contenido re-crawleado).
- **Objetivo:** encontrar todos los pares de documentos con alto contenido compartido.
- La comparación directa requiere $\mathcal{O}(n^2)$ comparaciones $\Rightarrow$ inviable a escala.

Ejemplo: la misma noticia en dos medios

Medio A: "El presidente anunció un nuevo plan económico en la cumbre de Bruselas."

Medio B: "El presidente presentó un nuevo plan económico en la cumbre de Bruselas."

Mismo hecho, texto casi idéntico $\Rightarrow$ *casi duplicados*.

¿Por qué es relevante?

- **Entrenamiento de LLMs:** los corpus web se deduplican casi-exactamente (Common Crawl → C4, The Pile). *Lee et al., ACL 2022*:
  - near-duplicados del **3,04 %** (C4) al **13,63 %** (RealNews).
  - datasets hasta **19 %** más pequeños ⇒ menos cómputo.
  - reduce la emisión memorizada **10**× y el solapamiento train–test (**>4 %** del set de validación).
  - una sola oración de 61 palabras repetida **>60 000 veces** en C4.
- **Web / crawlers:** ≈40 % de las páginas cambia en una semana (Cho & Garcia-Molina, VLDB 2000); a escala de Common Crawl (petabytes), deduplicar ahorra almacenamiento, cómputo y energía.
- **Otros dominios:** plagio y derecho (*containment*), spam/phishing e indexación en motores de búsqueda.

¿Qué es un *shingle*?

- Un *shingle* es un fragmento contiguo de k tokens (o de k caracteres): es similar a una **ventana deslizante** de tamaño k que avanza de a un token.
- Un documento se convierte en el **conjunto** de todos sus shingles $\Rightarrow$ se compara *contenido*, no texto literal.

Ejemplo con "el presidente anunció un plan económico" y $k = 3$:

$$S = \{\text{“el presidente anunció”}, \text{“presidente anunció un”}, \\ \text{“anunció un plan”}, \text{“un plan económico”}\}$$

Ventaja: robusto a cambios de puntuación, mayúsculas y espacios.

Formalización

Cada documento d_i se representa por su conjunto de k-shingles S_i.

Resemblance (Jaccard):

$$r(d_i, d_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}$$

Containment (inclusión):

$$c(d_i, d_j) = \frac{|S_i \cap S_j|}{|S_i|}$$

Dado un umbral $t \in (0, 1)$, encontrar

$$\mathcal{P}_t = \{(i, j) : i < j,\ r(d_i, d_j) \geq t\}$$

¿Por qué es un desafío?

- **Costo cuadrático:** $\frac{n(n-1)}{2}$ comparaciones; con $n = 18\,800$ son ≈ 177 millones de pares.
- **Memoria:** $O(n^2)$.
- **Representación:** el texto exige *shingling* sensible a longitud y ruido.
  - longitud: distinguir *resemblance* (similitud) de *containment* (inclusión).
- **Escala real:** la web reúne cientos de miles de millones de páginas (Common Crawl).

Solución existente

Comparación de todos los pares

- Jaccard **exacto** de cada par.
- Correcta $\Rightarrow$ *ground truth*.
- $O(n^2)$ en tiempo y memoria.

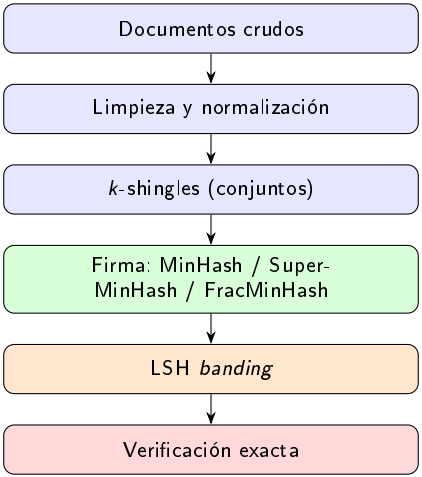
MinHash + LSH

- Firmas cortas + índice de candidatos.
- Sub-cuadrática, **aproximada**.
- Estándar industrial (C4, The Pile).

Usamos la primera como **baseline** y la segunda como **estado del arte**.

(Broder, 1997; Indyk & Motwani, 1998)

Idea de solución propuesta



- **Dos etapas:** firmas (reducen dimensiones) y LSH (reducen pares candidatos).
- Comparamos **tres esquemas de firma**; verificación **exacta** solo sobre candidatos.

¿Por qué es sub-cuadrático? Complejidad

Notación: n documentos, T tokens totales, m tamaño de firma, b bandas, C pares candidatos.

	Tiempo	**Memoria**
Baseline (todos los pares)	$\Theta(n^2)$	$\Theta(n^2)$
Propuesta (firmas + LSH)	$O(T + nm + nb + C\bar{s})$	$O(nm)$

- **Memoria:** lineal en n ($O(nm)$) frente a $O(n^2)$.
- **Tiempo:** sub-cuadrático si $C \ll \binom{n}{2}$ (LSH solo evalúa candidatos).
- **Salvedad:** *output-sensitive* — si hay $\Theta(n^2)$ pares reales, emitirlos ya es cuadrático; y LSH es aproximada (falsos negativos $\Rightarrow$ se mide *recall*).

Datos

- **20 Newsgroups** ($\approx$ 18 800 documentos, 20 categorías): corpus real con duplicados y mensajes cruzados.
- Ground truth **exacto** calculable $\Rightarrow$ permite medir *recall*.
- Preprocesamiento: normalizar, quitar cabeceras/firmas y generar shingles.

Repositorios:

- `http://qwone.com/~jason/20Newsgroups/`
- `https://scikit-learn.org/stable/datasets/real_world.html`

(Escala web real, *trabajo futuro*: Common Crawl — `https://commoncrawl.org/`)

Paper a presentar

SuperMinHash — A New Minwise Hashing Algorithm for Jaccard Similarity Estimation

- **Autor:** Otmar Ertl
- **Venue:** arXiv preprint `arXiv:1706.05698` (cs.DS)
- **Año:** 2017

- Mejora sobre MinHash: **más rápido** ($O(n + m \log^2 m)$) y **más preciso** (varianza $\frac{J(1-J)}{m}\alpha(m, u)$).
- Base de nuestra firma principal (comparada con MinHash y FracMinHash).

Trabajo relacionado: ProbMinHash (O. Ertl, *IEEE TKDE*, 2020).

Planificación

Avance

- Datos, shingling y **baseline exacto**.
- MinHash + SuperMinHash (interfaz común).
- Validación del estimador de Jaccard.
- Comparación preliminar MinHash vs SuperMinHash.

Entrega final

- LSH *banding* + calibración de la curva-S.
- FracMinHash y evaluación de *containment*.
- Experimentos end-to-end (P/R/F1, speedup).
- Escalabilidad y análisis crítico.
- Informe final + repositorio GitHub.

Referencias I


A. Z. Broder. *On the Resemblance and Containment of Documents*. Compression and Complexity of Sequences (SEQUENCES), 1997.


P. Indyk, R. Motwani. *Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality*. STOC, 1998.


O. Ertl. *SuperMinHash — A New Minwise Hashing Algorithm for Jaccard Similarity Estimation*. arXiv:1706.05698, 2017.


O. Ertl. *ProbMinHash — A Class of Locality-Sensitive Hash Algorithms for the (Probability) Jaccard Similarity*. IEEE TKDE, 2020.


J. Cho, H. Garcia-Molina. *The Evolution of the Web and Implications for an Incremental Crawler*. VLDB, 2000.


K. Lee et al. *Deduplicating Training Data Makes Language Models Better*. ACL, 2022.